COLLEGE EVENT REGISTRATION PORTAL

Hackathon – Fundamentals of Web Development
Level 2 Project Report

Student Name	Dharshini MS (document author; please verify)
Register Number	_____
Department	_____
College	_____
Date of Submission	29 October 2026
GitHub Link	_____

Prepared using the supplied project format and the supplied HTML, CSS, and JavaScript source-code PDF. The complete source-code PDF is appended after this report.

Contents

1	Problem Statement
2	Objectives
3	Architecture
4	Practical Implementation
4.1	Features Implemented
4.2	Source Code Overview
4.3	Project Execution
5	Output Screenshots / Interface Evidence
6	Technical Explanation
7	Conclusion
8	Future Enhancements
Appendix A	Complete supplied source-code PDF

Project summary

The College Event Registration Portal is a browser-based web application that lets students browse upcoming college events, filter events by name/category/date, register with their name and email, view registrations, and cancel a registration. The supplied implementation uses HTML for structure, CSS for responsive presentation, JavaScript for application logic, and browser localStorage for persistence on the same browser.

Scope note: this is a front-end demonstration. The supplied code does not implement a server, database, authentication, or cross-device synchronization.

1. Problem Statement

College events such as technical symposiums, cultural programmes, sports meets, and workshops often require students to discover event details and submit registrations. When information and sign-ups are handled manually, students may have difficulty finding current details, and organizers may need to check registrations and remaining capacity by hand.

The College Event Registration Portal provides a single web page for viewing available events and managing registrations. It presents event dates, venues, categories, descriptions, capacities, and seat availability. It also checks duplicate registrations for the same event/email and prevents registration when the event capacity has been reached.

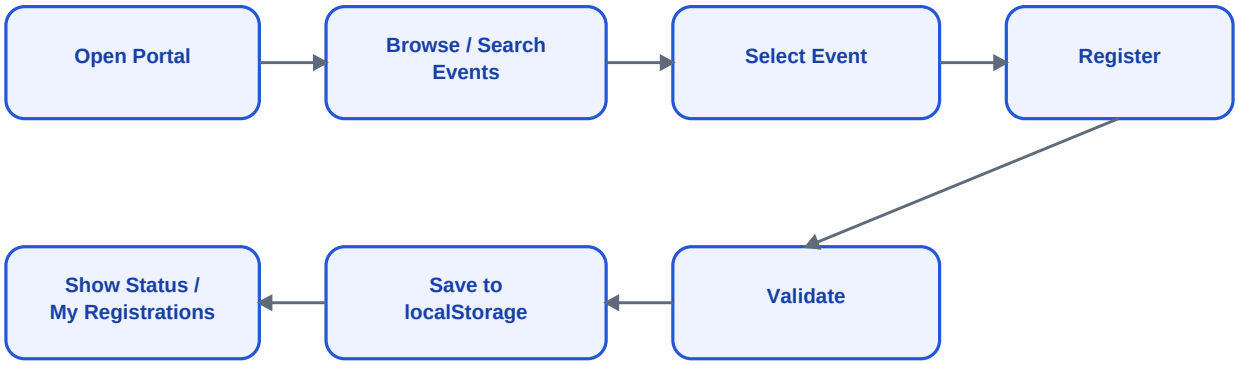
2. Objectives

- Present upcoming college events in a clear, card-based layout.
- Allow searching by event title and filtering by category or exact date.
- Show event information and registered/available seat counts.
- Collect a student's name and email through a registration form.
- Prevent duplicate registrations for the same event and email address.
- Enforce the capacity limit in the browser application.
- Persist registrations in localStorage and show a list of registrations.
- Allow users to cancel registrations and update the displayed counts.

3. Architecture

The application is a client-side web page made from three main files. The browser loads the HTML page, applies the stylesheet, and runs the JavaScript after the document is parsed. Event data is defined in JavaScript; registrations are read from and written to the browser's localStorage.

Application flow



Registration checks: duplicate email + event capacity

Technology stack

Technology	Role in the project
HTML5	Defines the header, navigation, event list, filters, registration form, registration list, and footer.
CSS3	Controls colors, spacing, event-card layout, form styling, feedback messages, and responsive behavior for narrow screens.
JavaScript	Defines event data, filters events, creates event cards, validates registrations, handles cancellations, and updates the page.
Browser localStorage	Stores registration records as JSON under the key `registrations` in the current browser.

Data flow: user action → JavaScript event handler → validation and registration array update → localStorage save → refresh event cards and registration list.

4. Practical Implementation

The project separates content, styling, and behavior into three files referenced by the HTML page: `index.html`, `style.css`, and `script.js`. The supplied source-code PDF contains the full code for these sections and is attached at the end of this report.

4.1 Features Implemented

Event catalogue	Four sample events are defined: Technical Symposium, Cultural Fest, Sports Meet, and Web Development Workshop.
Search and filters	Search matches the event title without case sensitivity; category and date filters can be combined.
Event details	The View Details action displays event date, venue, category, description, and capacity.
Registration form	The selected event ID is stored in a hidden field; student name and email are required inputs.
Duplicate check	A registration is rejected if the same email is already registered for the same event.
Capacity check	The Register button is disabled when available seats are zero, and submission checks capacity again.
My Registrations	Saved registrations display event title, student name, email, date, and a cancel button.
Cancellation and messages	Cancellation asks for confirmation; success and error messages are displayed in the page.
Responsive styling	CSS grid adapts event cards to available width; a media query adjusts layout on smaller screens.

SOURCE CODE

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>College Event Registration Portal</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>

<header>
  <h1>College Event Registration Portal</h1>
  <p>Explore Events &bull; Register &bull; Manage Your Registrations</p>
</header>

<nav>
  <a href="#events">Events</a>
  <a href="#myRegistrations">My Registrations</a>
</nav>

<div class="container">
  <h2 id="events">Upcoming College Events</h2>

  <div class="filters">
    <input type="text" id="search"
        placeholder="Search event name..."
        oninput="displayEvents()">

    <select id="category" onchange="displayEvents()">
      <option value="">All Categories</option>
      <option value="Technical">Technical</option>
      <option value="Cultural">Cultural</option>
      <option value="Sports">Sports</option>
    </select>

    <input type="date" id="dateFilter"
        onchange="displayEvents()">
  </div>
```

```
  <div id="eventList" class="event-list"></div>
  <div id="message"></div>

  <div class="section" id="registrationForm" style="display:none;">
    <h2>Event Registration</h2>

    <form id="regFormElement" onsubmit="registerEvent(event)">
      <input type="hidden" id="eventId">

      <p>
        <label for="studentName"><b>Student Name</b></label><br>
        <input type="text" id="studentName"
               placeholder="Enter your name"
               required class="full-width">
      </p>

      <p>
        <label for="studentEmail"><b>Email Address</b></label><br>
        <input type="email" id="studentEmail"
               placeholder="Enter your email"
               required class="full-width">
      </p>

      <button type="submit">Register Now</button>
      <button type="button" onclick="closeForm()" class="close-btn">
        Close
      </button>
    </form>
  </div>

  <div class="section" id="myRegistrations">
    <h2>My Registrations</h2>
    <div id="registeredEvents"></div>
  </div>
</div>

<footer>
  <p>&copy; 2026 College Event Registration Portal</p>
</footer>

</body>
</html>
```

CSS

```
* {
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  margin: 0;
  background: #f2f5fa;
  color: #333;
}

header {
  background: linear-gradient(135deg, #2457d6, #173fa5);
  color: white;
  text-align: center;
  padding: 30px 20px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}

header h1 {
  margin: 0;
  font-size: 2rem;
}

header p {
  margin-top: 8px;
  font-size: 0.95rem;
  opacity: 0.9;
}

nav {
  background: #173fa5;
  padding: 12px;
  text-align: center;
  box-shadow: inset 0 -2px 5px rgba(0,0,0,0.2);
}

nav a {
  color: white;
  text-decoration: none;
  margin: 0 15px;
  font-weight: bold;
}

nav a:hover {
  color: #a0c4ff;
}

.container {
  width: 90%;
  max-width: 1100px;
  margin: 25px auto;
}

.filters {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin-bottom: 25px;
```

```
    background: white;
    padding: 15px;
    border-radius: 10px;
    box-shadow: 0 3px 10px rgba(0,0,0,0.05);
}

input, select, button {
    padding: 10px 12px;
    border: 1px solid #ccc;
    border-radius: 5px;
    font-size: 0.95rem;
}

input, select {
    flex: 1;
    min-width: 150px;
}

.full-width {
    width: 100%;
}

button {
    background: #2457d6;
    color: white;
    border: none;
    cursor: pointer;
    font-weight: bold;
    transition: background 0.2s;
}

button:hover {
    background: #173fa5;
}

button:disabled {
    background: #b0b0b0;
    cursor: not-allowed;
}

.event-list {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 20px;
}

.event-card {
    background: white;
    padding: 20px;
    border-radius: 10px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.08);
    border-top: 5px solid #2457d6;
}

.event-card h3 {
    color: #2457d6;
    margin-top: 0;
}
```

```
.event-card p {
  line-height: 1.5;
  margin: 8px 0;
}

.event-card button {
  margin: 8px 5px 0 0;
}

.details {
  background: #eef3ff;
  padding: 12px;
  border-radius: 6px;
  margin: 12px 0;
  border-left: 4px solid #2457d6;
}

.details p {
  margin: 4px 0;
  font-size: 0.9rem;
}

.section {
  background: white;
  padding: 25px;
  margin-top: 30px;
  border-radius: 10px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.08);
}

.registered-item {
  border-bottom: 1px solid #eee;
  padding: 15px 0;
}

.cancel-btn {
  background: #dc3545;
}

.cancel-btn:hover {
  background: #b02a37;
}

.close-btn,
.close-btn:hover {
  background: #6c757d;
}

footer {
  text-align: center;
  background: #173fa5;
  color: white;
  padding: 20px;
  margin-top: 40px;
}

#message {
  font-weight: bold;
  margin: 15px 0;
  padding: 12px;
```

```
    border-radius: 5px;
    display: none;
}

.success-msg {
    display: block !important;
    background-color: #d4edda;
    color: #155724;
    border: 1px solid #c3e6cb;
}

.error-msg {
    display: block !important;
    background-color: #f8d7da;
    color: #721c24;
    border: 1px solid #f5c6cb;
}

@media (max-width: 600px) {
    header h1 {
        font-size: 1.5rem;
    }

    nav a {
        margin: 0 8px;
    }

    .container {
        width: 94%;
    }

    .section {
        padding: 18px;
    }
}
```

JavaScript

```
// Event Data
let events = [
  {
    id: 1,
    title: "Technical Symposium",
    date: "2026-11-15",
    venue: "Computer Science Department",
    description: "Technical quiz, coding and paper presentation.",
    category: "Technical",
    capacity: 50
  },
  {
    id: 2,
    title: "Cultural Fest",
    date: "2026-11-20",
    venue: "College Auditorium",
    description: "Dance, music and cultural performances.",
    category: "Cultural",
    capacity: 100
  },
  {
    id: 3,
    title: "Sports Meet",
    date: "2026-11-25",
    venue: "College Ground",
    description: "Indoor and outdoor sports competitions.",
    category: "Sports",
    capacity: 60
  },
  {
    id: 4,
    title: "Web Development Workshop",
    date: "2026-12-05",
    venue: "Computer Lab",
    description: "Learn HTML, CSS and JavaScript.",
    category: "Technical",
    capacity: 30
  }
];

// Load Registrations
let registrations =
  JSON.parse(localStorage.getItem("registrations")) || [];

// Save Registrations
function saveRegistrations() {
  localStorage.setItem(
    "registrations",
    JSON.stringify(registrations)
  );
}

// Display Events
function displayEvents() {
  let search = document.getElementById("search").value.toLowerCase();
  let category = document.getElementById("category").value;
  let date = document.getElementById("dateFilter").value;
```

```
  let eventList =
document.getElementById("eventList");

  eventList.innerHTML = "";

  let filteredEvents = events.filter(function(event) {
    return event.title.toLowerCase().includes(search)
&&
      (category === "" || event.category === category)
&&
      (date === "" || event.date === date);
  });

  if (filteredEvents.length === 0) {
    eventList.innerHTML = "<p>No events found.</p>";
    return;
  }

  filteredEvents.forEach(function(event) {
    let count = registrations.filter(
      reg => reg.eventId === event.id
    ).length;

    let available = event.capacity - count;
    let card = document.createElement("div");

    card.className = "event-card";

    card.innerHTML = `
      <h3>${event.title}</h3>
      <p><b>Date:</b> ${event.date}</p>
      <p><b>Venue:</b> ${event.venue}</p>
      <p><b>Category:</b> ${event.category}</p>
      <p>${event.description}</p>

      <div class="details">
        <p><b>Registered:</b> ${count}</p>
        <p><b>Available Seats:</b> ${available}</p>
        <p><b>Capacity:</b> ${event.capacity}</p>
      </div>

      <button onclick="showDetails(${event.id})">
        View Details
      </button>

      <button onclick="openForm(${event.id})"
        ${available <= 0 ? "disabled" : ""}>
        ${available <= 0 ? "Event Full" : "Register"}
      </button>
    `;

    eventList.appendChild(card);
  });
}

// Show Event Details
function showDetails(id) {
  let event = events.find(e => e.id === id);

  if (!event) return;

  alert(
    "Event: " + event.title +
```

```
      "\: " + event.date +
      "\nVenue: " + event.venue +
      "\nCategory: " + event.category +
      "\nDescription: " + event.description +
      "\nCapacity: " + event.capacity
    );
}

// Open Registration Form
function openForm(id) {
    document.getElementById("eventId").value = id;
    document.getElementById("registrationForm").style.display = "block";

    document.getElementById("registrationForm").scrollIntoView({
        behavior: "smooth"
    });
}

// Close Registration Form
function closeForm() {
    document.getElementById("registrationForm").style.display = "none";
    document.getElementById("regFormElement").reset();
}

// Register for Event
function registerEvent(event) {
    event.preventDefault();

    let id = Number(document.getElementById("eventId").value);
    let name = document.getElementById("studentName").value.trim();
    let email = document.getElementById("studentEmail")
        .value.trim().toLowerCase();

    let selectedEvent = events.find(e => e.id === id);

    if (!selectedEvent) return;

    // Check Duplicate Registration
    let duplicate = registrations.some(
        reg => reg.eventId === id && reg.email === email
    );

    if (duplicate) {
        showMessage(
            "You have already registered for this event!",
            "error"
        );
        return;
    }

    // Check Event Capacity
    let count = registrations.filter(
```

```
        reg => reg.eventId === id
    ).length;

    if (count >= selectedEvent.capacity) {
        showMessage("Sorry! This event is full.", "error");
        return;
    }

    // Save New Registration
    let registration = {
        id: Date.now(),
        eventId: id,
        name: name,
        email: email
    };

    registrations.push(registration);
    saveRegistrations();

    showMessage("Registration successful!", "success");

    closeForm();
    displayEvents();
    displayRegistrations();
}

// Display Registered Events
function displayRegistrations() {
    let container = document.getElementById("registeredEvents");
    container.innerHTML = "";

    if (registrations.length === 0) {
        container.innerHTML = "<p>No registrations yet.</p>";
        return;
    }

    registrations.forEach(function(reg) {
        let event = events.find(e => e.id === reg.eventId);

        if (!event) return;

        let item = document.createElement("div");
        item.className = "registered-item";

        item.innerHTML = `
            <h3>${event.title}</h3>
            <p><b>Student:</b> ${reg.name}</p>
            <p><b>Email:</b> ${reg.email}</p>
            <p><b>Date:</b> ${event.date}</p>

            <button class="cancel-btn"
                onclick="cancelRegistration(${reg.id})">
                Cancel Registration
            </button>
        `;

        container.appendChild(item);
    });
}
```

```
// Cancel Registration
function cancelRegistration(id) {
    if (!confirm("Are you sure you want to cancel this registration?")) {
        return;
    }

    registrations = registrations.filter(reg => reg.id !== id);
    saveRegistrations();

    showMessage("Registration cancelled successfully!", "success");

    displayEvents();
    displayRegistrations();
}

// Display Messages
function showMessage(message, type = "") {
    let msgElement = document.getElementById("message");

    msgElement.innerText = message;
    msgElement.className = "";

    if (type === "success") {
        msgElement.classList.add("success-msg");
    } else if (type === "error") {
        msgElement.classList.add("error-msg");
    }
}

// Initial Calls
displayEvents();
displayRegistrations();
```

4.3 Project Execution

Requirements

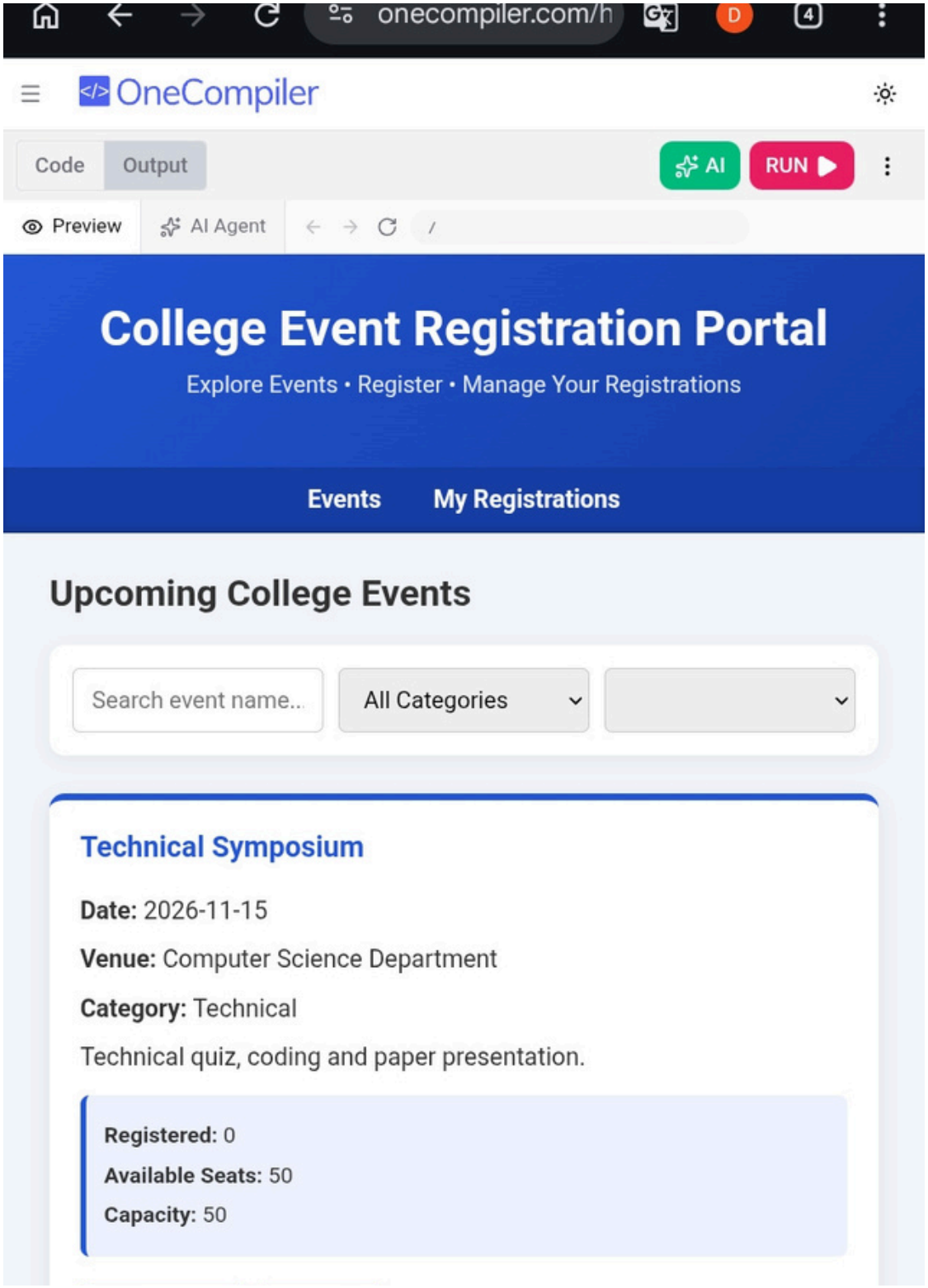
A modern web browser is sufficient. No package installation or server is required for the supplied static version.

Steps to run

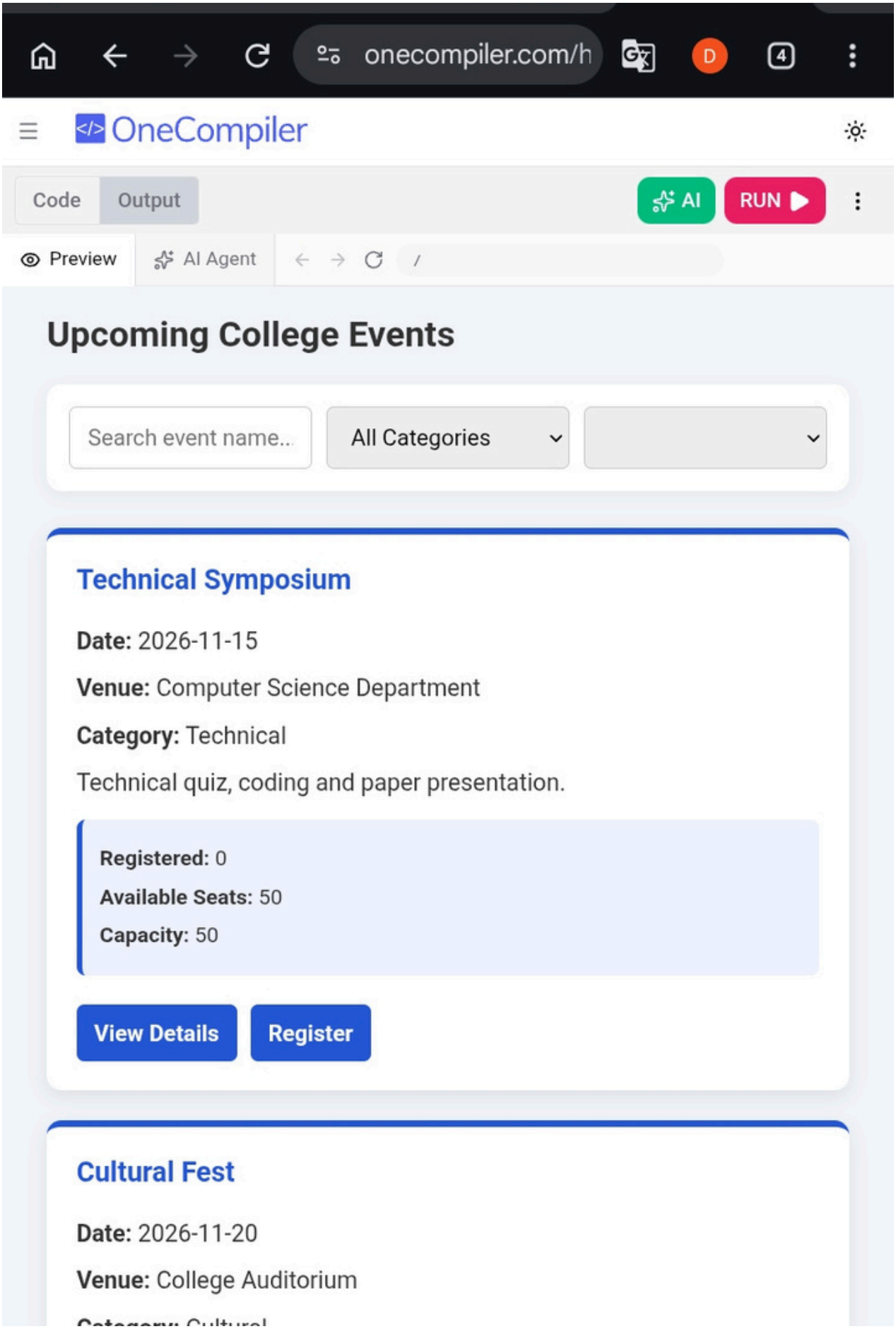
1. Create a folder for the project.
2. Save the supplied HTML as `index.html`, the CSS as `style.css`, and the JavaScript as `script.js` in the same folder.
3. Ensure the HTML contains the stylesheet link and the deferred script link shown in the source.
4. Open `index.html` in a modern browser.
5. Search or filter events, choose Register, enter a name and email, and submit the form.
6. Scroll to My Registrations to review or cancel a registration. Refreshing the page should retain registrations in that same browser unless local storage is cleared.

Expected working process

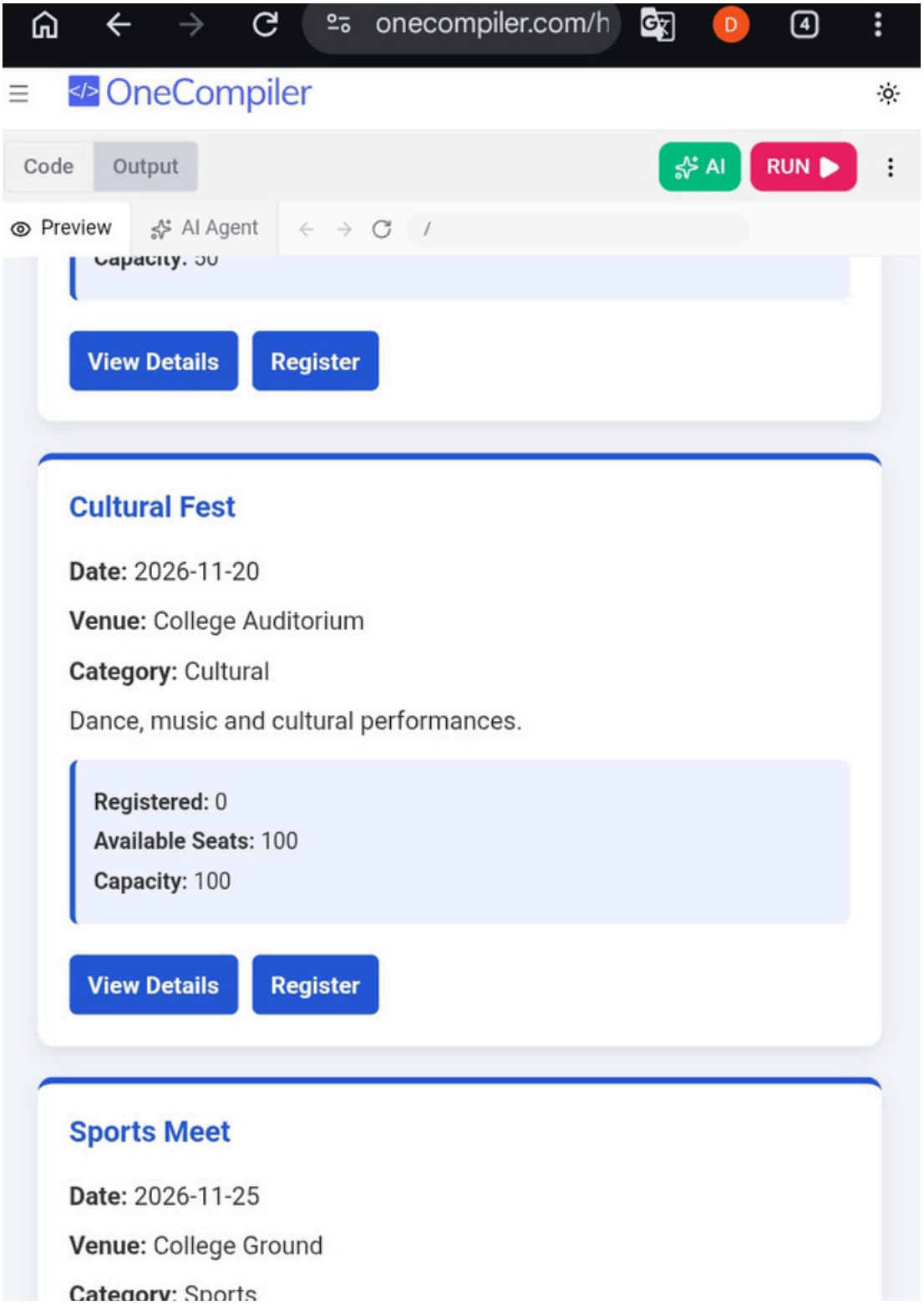
On page load, the script displays event cards and any saved registrations. Filtering re-renders the matching cards. Choosing Register opens the form for that event. Submitting a valid registration updates localStorage and refreshes seat counts and My Registrations. Duplicate registrations and full events are rejected with a message. Cancelling removes the selected registration after confirmation.



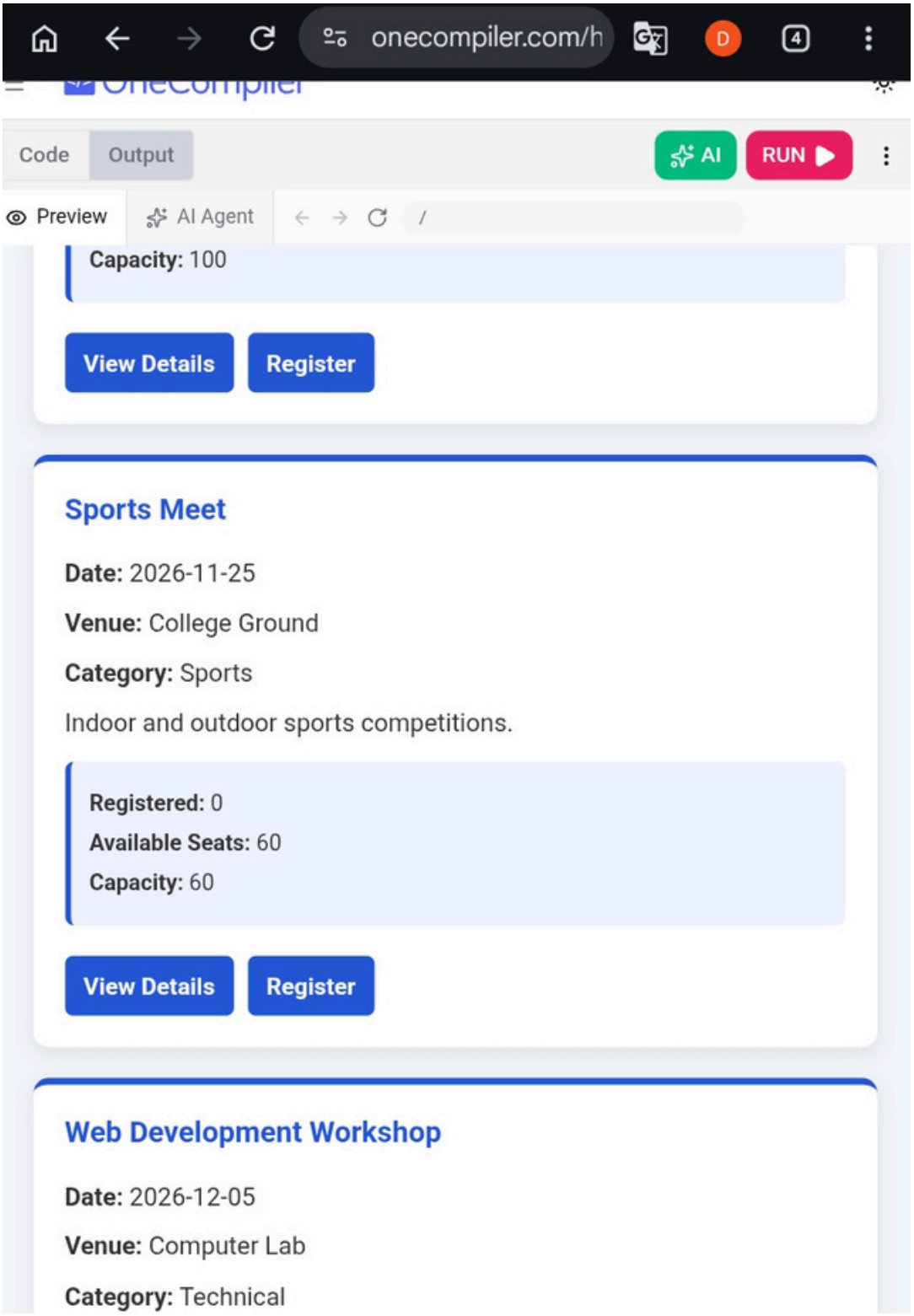
Screenshot 1: Home page of college event registration Portal



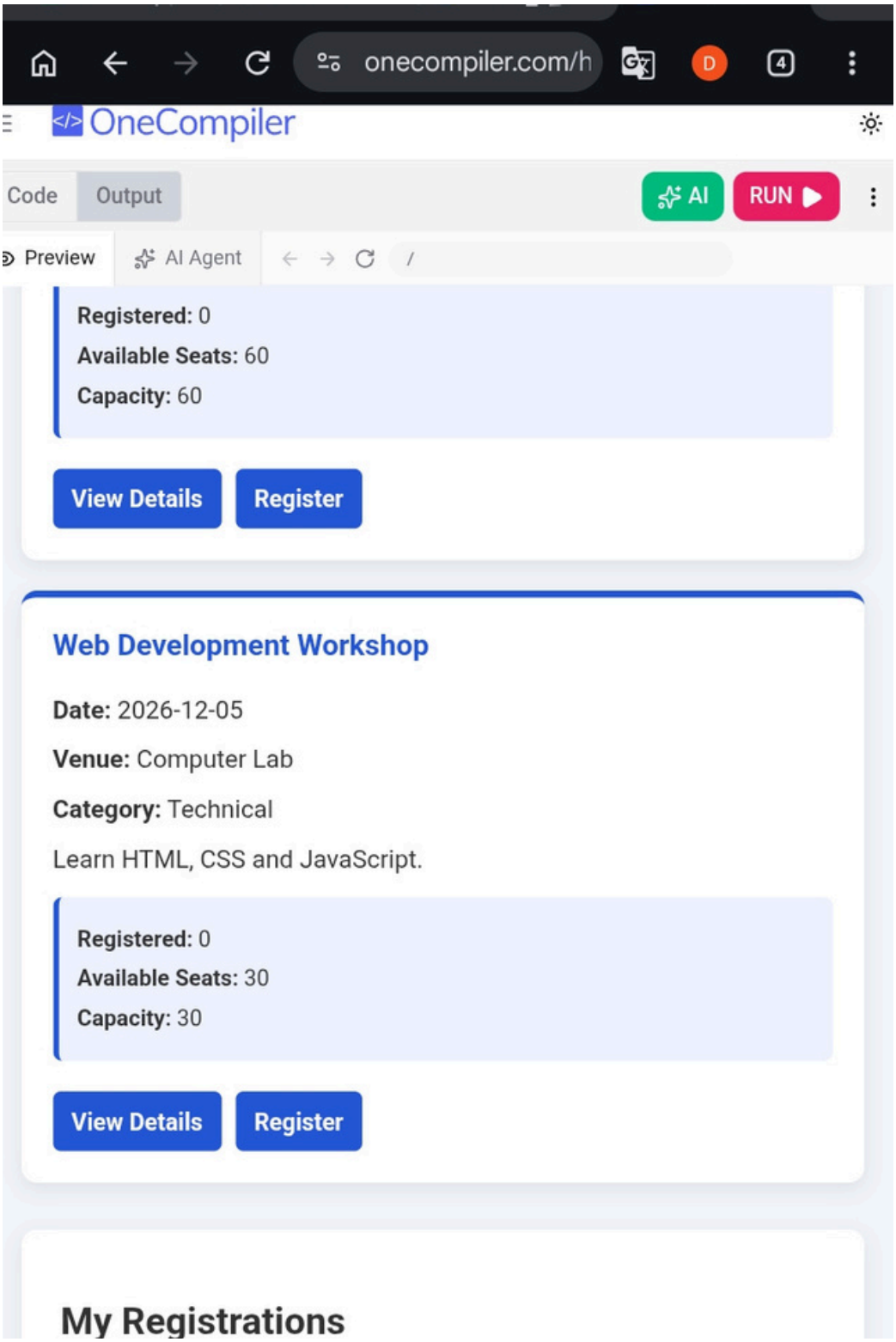
Screenshot 2: Technical Symposium



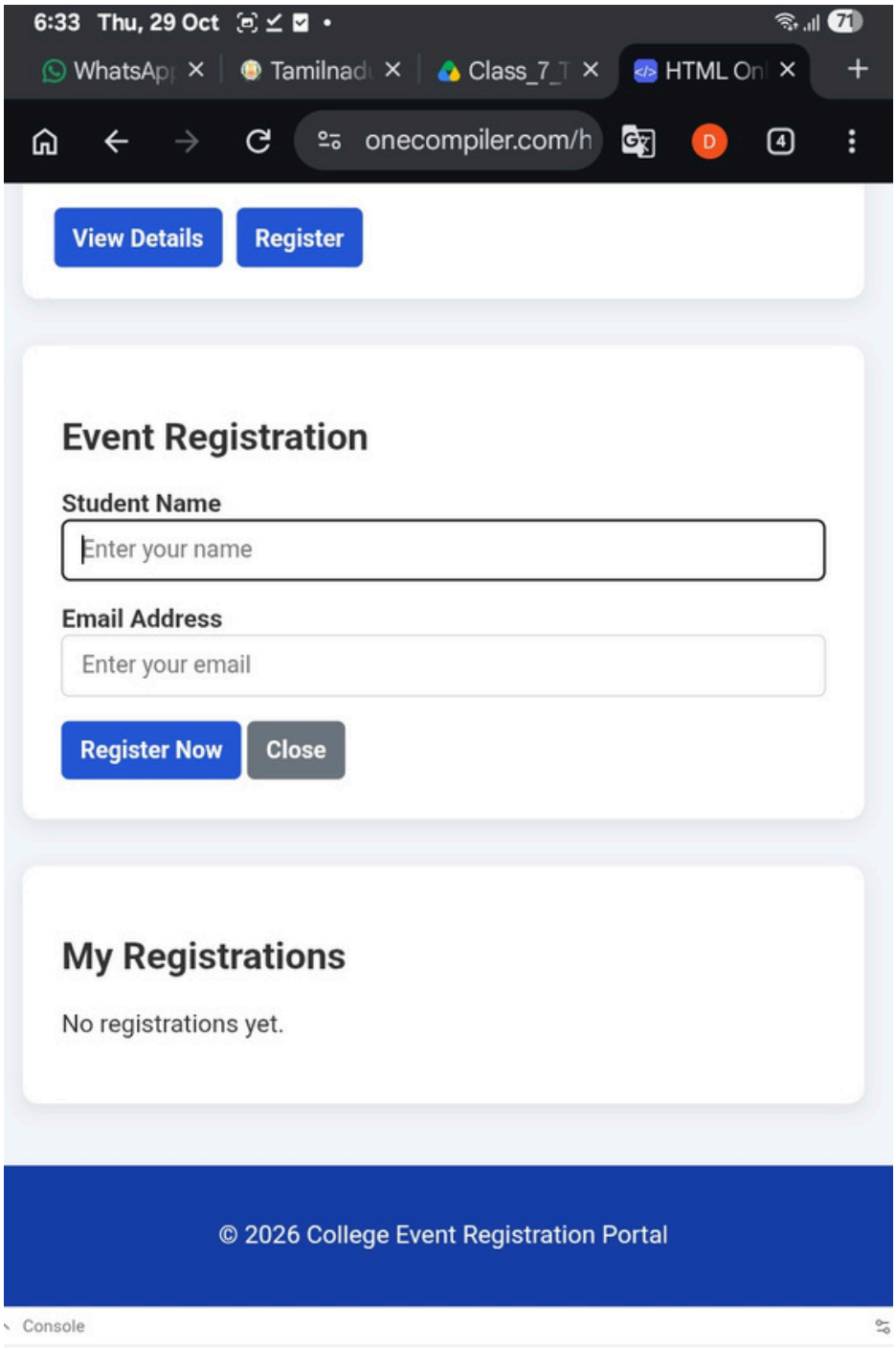
Screenshot 3: Cultural Fest



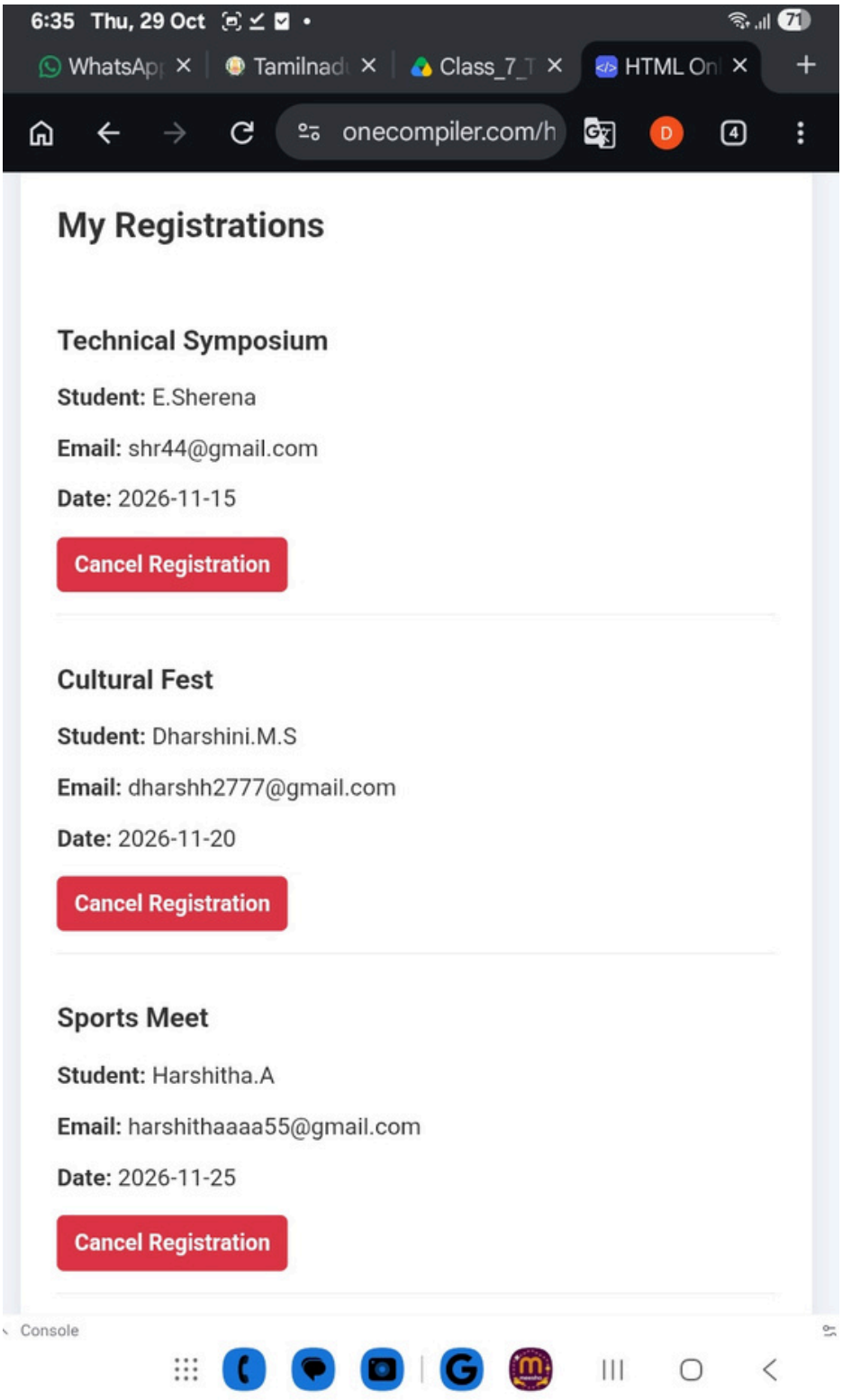
Screenshot 4: Sports Meet



Screenshot 5: Web Development Workshop



Screenshot 6: Event Registration



Screenshot 7: My Registrations

5. Output Screenshots / Interface Evidence

The supplied second PDF contains source code rather than captured screenshots of a running browser session. To avoid presenting a fabricated screenshot as real evidence, the diagram below is a representative interface layout based on the supplied HTML and CSS. Replace it with actual browser screenshots if your college requires proof of execution.



Suggested real screenshots to capture for final submission

1. Initial page showing all event cards and filters. 2. Registration form for a selected event. 3. Successful registration and updated seat count. 4. My Registrations with a cancel option. 5. Duplicate-registration or full-capacity validation message.

6. Technical Explanation

DOM manipulation: JavaScript reads controls with `document.getElementById()`, creates event-card elements, and updates the registration list dynamically.

Array operations: `filter()` selects matching events and removes cancelled registrations; `find()` locates an event; `some()` detects duplicates; `forEach()` renders lists.

Form handling: the submit handler calls `preventDefault()` to avoid a page reload, trims input, converts email to lowercase, and applies duplicate/capacity checks.

Persistence: registrations are serialized using `JSON.stringify()` and saved with `localStorage.setItem()`. On startup, `JSON.parse(localStorage.getItem("registrations"))` loads the saved data or uses an empty array.

Validation boundary: the browser checks improve usability but are not secure enforcement because all data and logic run on the client. Server-side checks would be required for a real multi-user portal.

7. Conclusion

The College Event Registration Portal demonstrates the fundamentals of web development by combining HTML structure, CSS presentation, and JavaScript behavior. It provides an event catalogue, filters, a registration form, duplicate and capacity checks, local persistence, registration listing, cancellation, and user feedback. The project is appropriate as a front-end learning prototype; a production deployment would need server-side storage and security.

8. Future Enhancements

- Add a backend API and a database so all students see shared, reliable registrations.
- Add student login, organizer/admin roles, and secure session management.
- Move duplicate and capacity validation to the server and use database transactions to avoid overbooking.
- Allow organizers to create, edit, publish, and close events.
- Send registration confirmations and cancellation notifications by email.
- Generate QR-code tickets and provide attendance check-in.
- Add accessible form labels, stronger input validation, automated tests, and clearer error states.
- Provide downloadable registration reports and event analytics for authorized organizers.

Appendix A — Complete supplied source-code PDF

The next section reproduces the second PDF supplied with the request, containing the complete HTML, CSS, and JavaScript source code. It is included as provided so that the report and source-code document remain together in one submission PDF.

Note: fill in the blank student information fields and add your actual GitHub URL before submission. Verify the student name shown on the cover.